Smart MCQ Solver

Multi-Model Analysis for Scientific Question Answering

Final Project Report

Deep Learning for GenAI

Kaggle Competition: Smart MCQ Solver Challenge

Best Public Leaderboard Score: MAP@3 = 0.75228

1. Abstract / Executive Summary

This project addresses the Kaggle "Smart MCQ Solver Challenge," a 5-option multiple choice question answering task over scientific domains (physics, biology, chemistry, astrophysics). The objective is to predict the top-3 most probable correct options for each prompt, evaluated via Mean Average Precision at 3 (MAP@3).

The experimental methodology followed a progressive approach, beginning with a traditional machine learning baseline and advancing towards deep learning and transformer-based architectures. Initially, a **TF-IDF + Logistic Regression** model was implemented to establish a strong baseline by representing the textual data as numerical feature vectors. Subsequently, an **Attention-based Bidirectional Long Short-Term Memory (BiLSTM)** network was developed to capture contextual and sequential dependencies within the question and answer options. Finally, a **pre-trained DeBERTa-v3-small transformer** was fine-tuned using the Hugging Face Transformers framework to leverage contextual language representations for improved multiple-choice question answering. Each model was trained and evaluated independently using the competition evaluation metric (MAP@3), along with Accuracy and Macro F1-score, enabling a comprehensive comparison of traditional machine learning, recurrent neural networks, and transformer-based approaches.

The experimental results demonstrated that the **fine-tuned DeBERTa-v3-small model** achieved the best overall performance among the three approaches, outperforming both the TF-IDF + Logistic Regression baseline and the Attention-based BiLSTM model due to its superior contextual understanding and pre-trained language representations. Furthermore, the final DeBERTa model was successfully deployed as an interactive web application using **Gradio on Hugging Face Spaces**, allowing users to input a question with five answer options and receive the predicted answer along with the top-3 ranked predictions.

.

2. Introduction

2.1. Problem Statement

The Kaggle competition required participants to solve 5-option scientific MCQs spanning physics, biology, chemistry, and astrophysics. For each question, the model must produce a ranked list of the top-3 most likely correct options (A, B, C, D, E), evaluated using MAP@3. Unlike standard multi-class classification with a single softmax, this task rewards partial credit: even if the top-1 prediction is wrong, placing the correct answer at rank 2 or 3 earns fractional MAP credit (1/2 or 1/3).

2.2. Project Objectives

- Maximize MAP@3: Achieve the highest possible ranking metric on the Kaggle Leaderboard by combining complementary model architectures.
- Comparative Model Evaluation: Systematically evaluate the performance of three different approaches for multiple-choice question answering: a traditional machine learning model (TF-IDF + Logistic Regression), a deep learning model (Attention-based BiLSTM), and a transformer-based language model (fine-tuned DeBERTa-v3-small). The comparison is performed using Accuracy, Macro F1-score, and MAP@3 to analyze the strengths and limitations of each approach.
- Technical Implementation: Develop and fine-tune the complete machine learning pipeline, including text preprocessing, TF-IDF vectorization, label encoding, tokenization, sequence padding, Attention-based BiLSTM architecture, and DeBERTa-v3-small fine-tuning using the Hugging Face Transformers framework. Additionally, deploy the final DeBERTa model as an interactive web application using Gradio on Hugging Face Spaces for real-time MCQ prediction.

-

3. Dataset & Preprocessing

3.1. Dataset Description

The provided dataset consists of 2,000 training samples and 500 test samples. Each sample contains a scientific prompt text and 5 candidate answer options (A through E), with exactly one correct answer. The prompts span multiple scientific domains including quantum mechanics, astrophysics, organic chemistry, ecology, and immunology.

3.2. Exploratory Data Analysis (EDA)

A thorough EDA revealed two critical structural insights about target label balance and prompt length distributions:

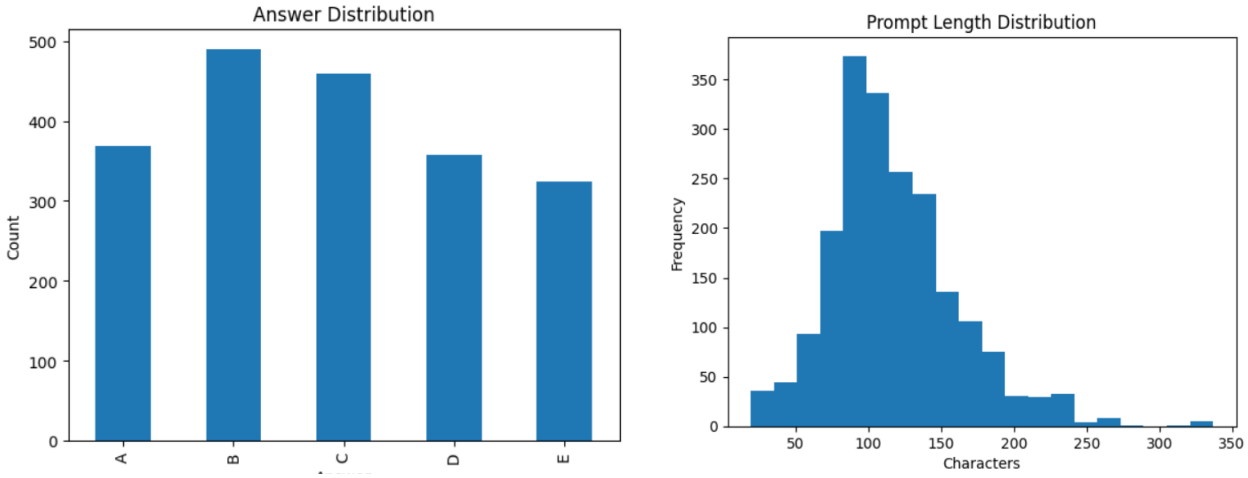


Figure 1: Answer Choice Frequency and Prompt Word Length Distribution

The slightly higher frequency of options B (24.5%) and C (22.9%) as correct responses suggests a little preference for the middle position when drafting options.The prompt length ranges from 19 to 337 characters, with a mean of approximately 118 characters. Most prompts are concentrated between 50 and 200 characters, indicating that a maximum token length of 256 provides sufficient coverage for the majority of inputs, while only a small number of longer sequences may require truncation.

3.3. Optional Discoveries

To better understand the textual characteristics of the dataset, exploratory analysis was performed on the question prompts. The Prompt Word Count Distribution indicates that most questions contain approximately 10–25 words**,** suggesting that the dataset primarily consists of moderately sized prompts, which supports the use of a fixed maximum sequence length during tokenization. Additionally, the Top 20 Most Frequent Words analysis reveals that common grammatical words such as *"the"*, *"of"*, *"a"*, and *"is"* occur most frequently, reflecting the natural language structure of the questions. Together, these analyses provide valuable insights into the dataset's length distribution and vocabulary, supporting the selection of appropriate text preprocessing and representation techniques, including TF-IDF vectorization and transformer-based tokenization.

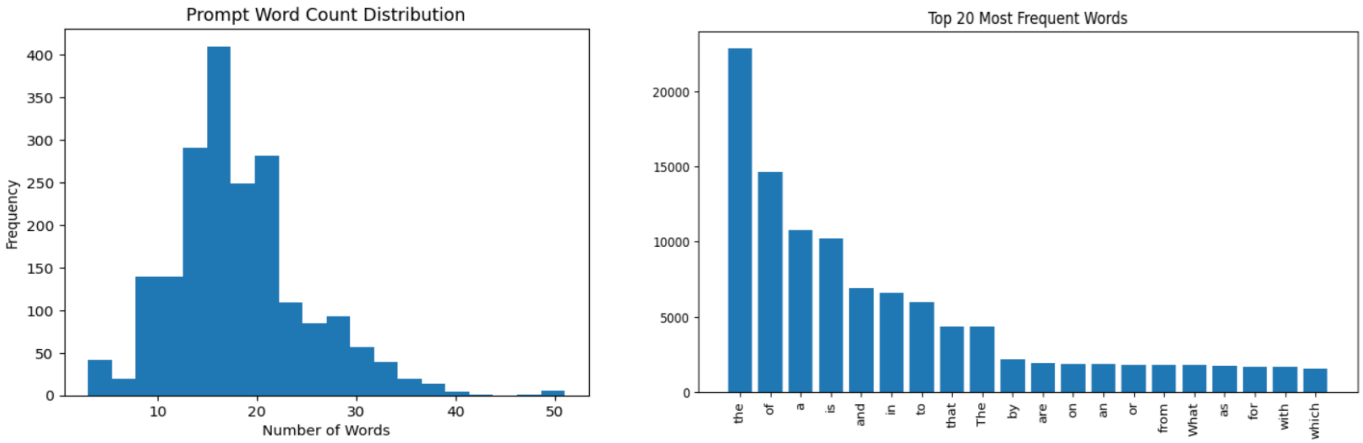


Figure 2: Optional Discoveries

4. Tokenization Strategy

4.1. TF-IDF Vectorization (Model 1)

For the traditional machine learning baseline, textual data was transformed into numerical feature vectors using **Term Frequency–Inverse Document Frequency (TF-IDF)** vectorization. The question prompt and all five answer options were concatenated into a single text sequence before vectorization. This approach converts textual information into a high-dimensional sparse feature matrix by assigning greater importance to informative terms while reducing the influence of frequently occurring words. The generated TF-IDF features were then used as input to the Logistic Regression classifier for multiple-choice answer prediction.

4.2. BERT Tokenization (Model 2)

The Attention-based BiLSTM model employed the AutoTokenizer associated with **bert-base-uncased** from the Hugging Face Transformers library. The combined question and answer text was tokenized using WordPiece subword tokenization, producing token IDs suitable for sequence modeling. Each input sequence was tokenized with a maximum sequence length of 128 tokens, applying truncation and fixed-length padding (`padding="max_length"`). The resulting tokenized sequences were subsequently processed by the embedding layer of the BiLSTM with Attention network.

4.3. DeBERTa-v3 Tokenization (Model 3)

The transformer-based model utilized the **AutoTokenizer** associated with **microsoft/deberta-v3-small**. This tokenizer employs SentencePiece subword tokenization to efficiently represent both common and rare words. Each combined question and answer sequence was tokenized with a maximum sequence length of **256 tokens**, while applying truncation and dynamic padding during training. The tokenized inputs were then used to fine-tune the DeBERTa-v3-small model using the Hugging Face Transformers framework for multiple-choice question answering.

5. Modeling & Experimentation

5.1. Model 1: TF-IDF + Logistic Regression (Model of choice)

The first model employs a traditional machine learning pipeline consisting of **TF-IDF feature extraction** followed by **Logistic Regression** for binary classification. Each multiple-choice question was converted into five question-option pairs, where the correct option was assigned a label of **1** and the remaining options were assigned **0**. Basic preprocessing, including lowercasing and punctuation removal, was applied before feature extraction.

The **TfidfVectorizer** was configured with **English stop-word removal**, **20,000 maximum features**, and **unigram–bigram (**ngram_range=(1,2)**)** representations to capture both individual words and short phrases. The resulting sparse TF-IDF vectors were used to train a Logistic Regression classifier with **500 maximum iterations**.

During experimentation, the dataset was split into **80% training** and **20% validation** while ensuring that all options belonging to the same question remained in the same split to prevent data leakage. After validation, the model was retrained on the complete training dataset and used to generate ranked predictions for the test set. Model performance was evaluated using **Accuracy**, **Macro F1-Score**, and **MAP@3**.

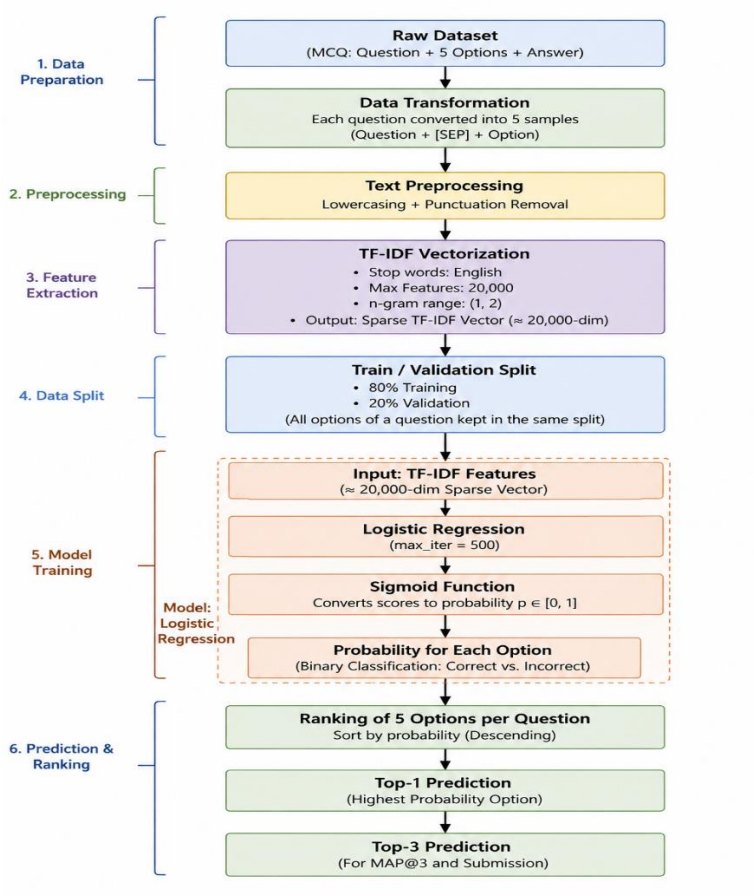


Figure 5.1.1: Architectural Diagram end to end

5.1.2 Results Table

Metric	Score
Accuracy	0.1950
Macro F1-Score	0.1849
MAP@3	0.3442

.



Figure 5.1.2: WandB - TF-IDF + Logistic Regression results

5.2. Model 2: BiLSTM + Attention (Scratch Model)

A custom **BiLSTM + Attention** model was implemented using PyTorch. Each question-option pair was tokenized using the bert-base-uncased tokenizer with a maximum sequence length of 128. The architecture consists of a trainable 100-dimensional embedding layer, a bidirectional LSTM with 128 hidden units per direction, an attention layer for learning token importance, dropout (0.3), and a final linear classification layer producing a binary logit for each question-option pair.

Training

The model was trained for **10 epochs** using the **Adam optimizer** with a learning rate of 1e-3 and BCEWithLogitsLoss. A batch size of 64 was used. During inference, sigmoid probabilities were generated for each of the five options, ranked by confidence, and the top-3 predictions were evaluated using MAP@3. The model configuration and training metrics were tracked using Weights & Biases. The notebook's W&B run records 10 epochs, Adam with learning rate 1e-3, and BCEWithLogitsLoss.

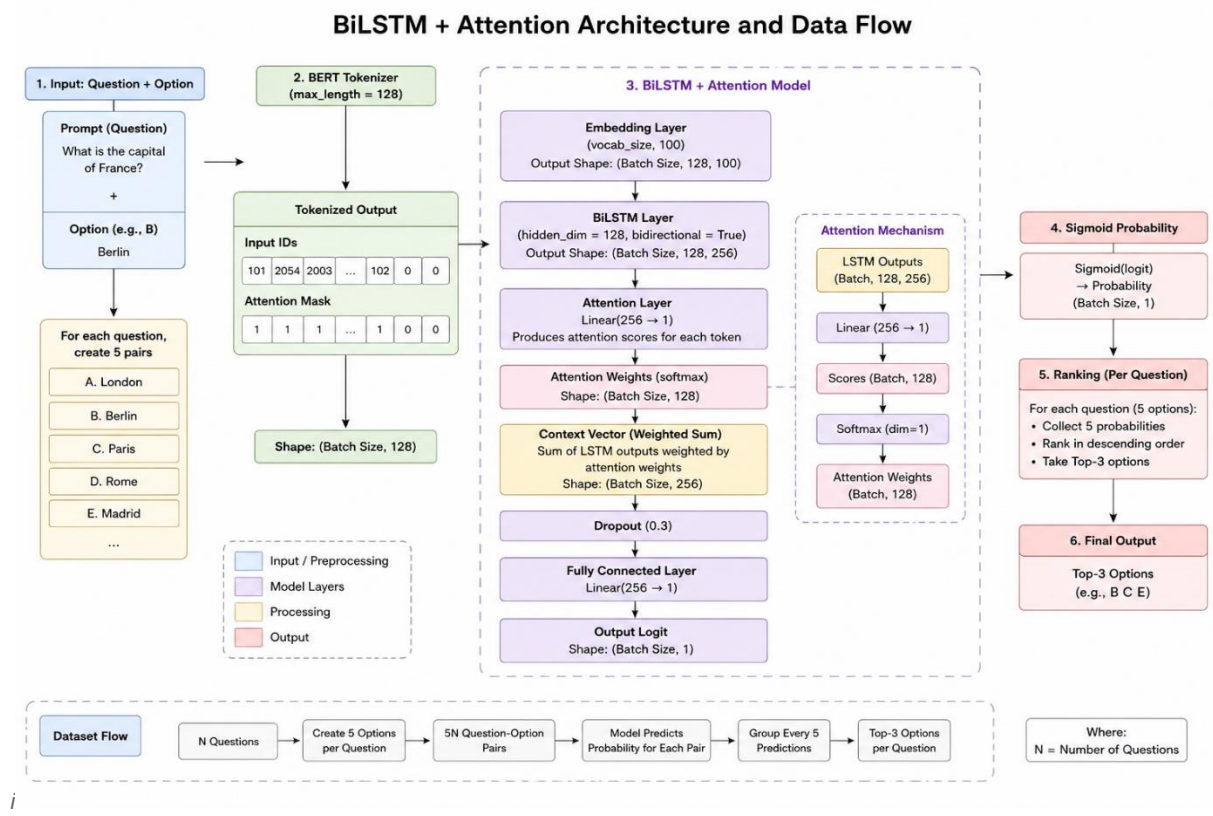


Figure 5.2.1: Architectural Diagram end to end

The BiLSTM captures contextual information from both forward and backward directions, while the attention mechanism assigns importance scores to individual token representations and aggregates them into a context vector. This representation is then used to estimate the probability that each option is the correct answer. The implementation uses nn.Embedding, bidirectional nn.LSTM,a linear attention scoring layer, dropout, and a final linear classifier.

Performance

Training Loss: 0.0277

Validation Loss: 0.02297

Validation Accuracy: 99.15%

Validation MAP@3: 0.9975

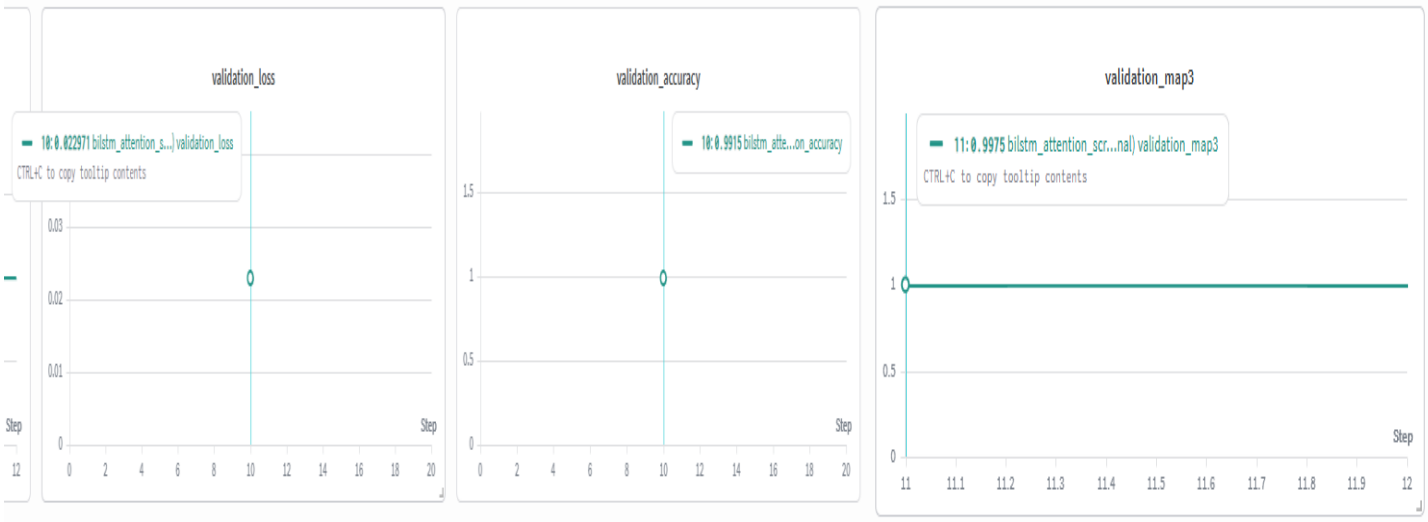


Figure 5.2.2: WandB – Validation loss, validation accuracy and validation map3 score

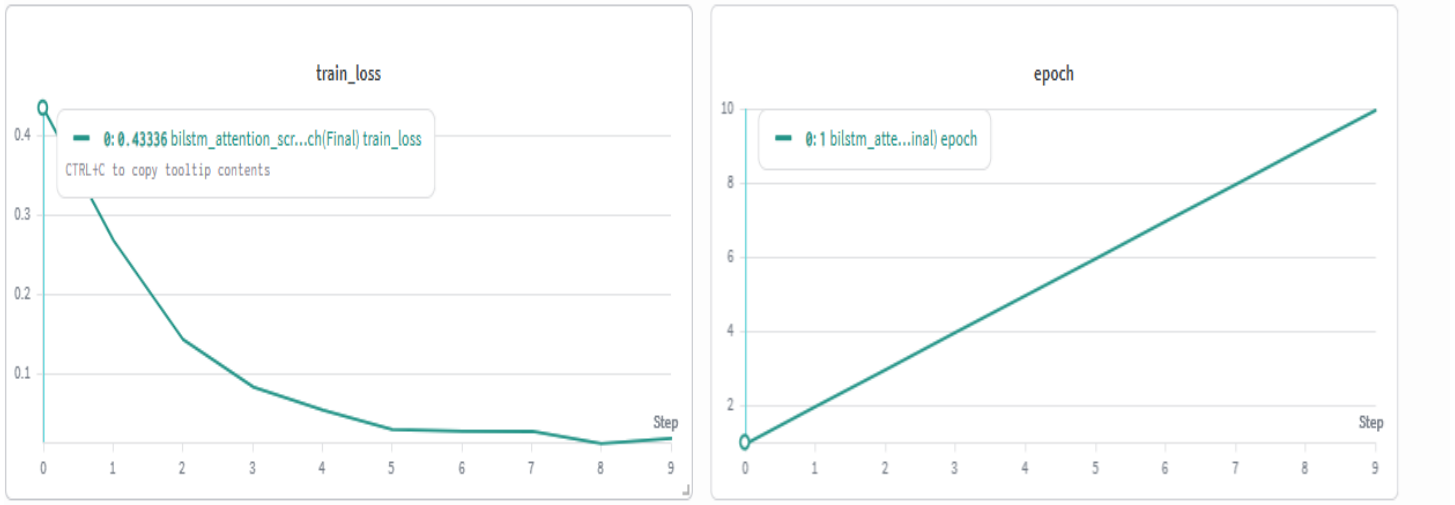


Figure 5.2.3: WandB – train_loss as per epochs results

5.3. Model 3:DeBerta (PreTrained Model)

A pretrained microsoft/deberta-v3-small model was fine-tuned as a **five-class MCQ classifier**. Each input contains the question together with all five answer options (A–E). The DeBERTa tokenizer converts the text into token IDs with a maximum sequence length of 256, followed by the pretrained Transformer encoder and a task-specific classification head producing five logits corresponding to the five answer choices.

Training

The model was fine-tuned for **5 epochs** using a learning rate of **2×10^{-5}**, batch size **16**, weight decay **0.01**, and FP16 training. Dynamic padding was applied using DataCollatorWithPadding, while early stopping with a patience of 2 was used to control unnecessary training. The best checkpoint was selected according to validation **MAP@3**.

Data Split

GroupShuffleSplit was used with prompt as the grouping variable to prevent the same prompt from appearing in both training and validation sets. This resulted in 1,609 training and 391 validation questions, with zero overlapping prompts between the two sets.

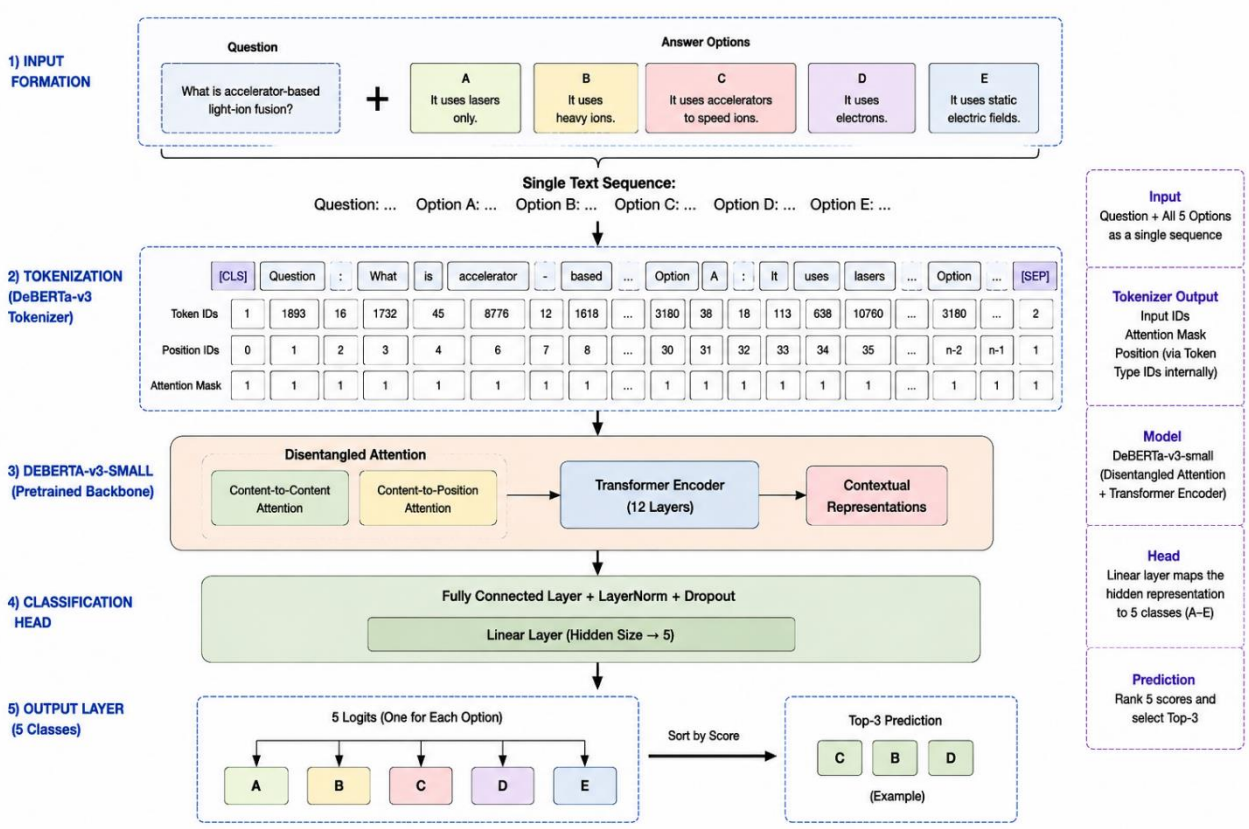


Figure 5.3.1: Architectural Diagram end to end

Performance

Training Loss: 0.0308

Validation Loss: 0.019943

Validation Accuracy: 99.7442%

Validation MAP@3: 99.8721%

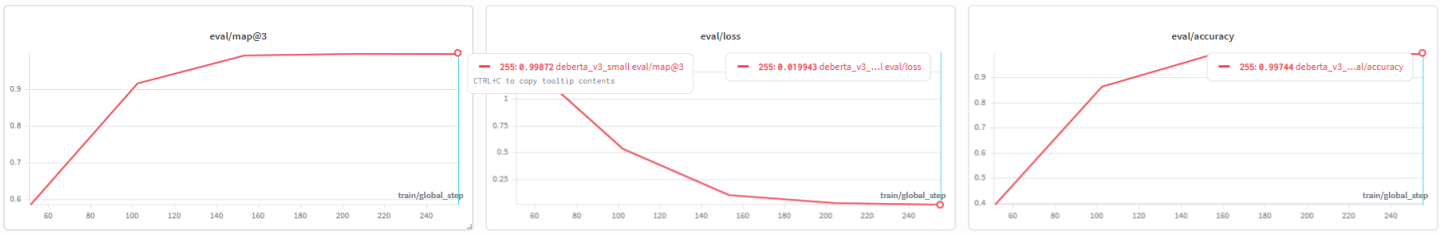


Figure 5.3.2: WandB – metrics map3,loss and accuracy

The DeBERTa-v3-small model demonstrated strong validation performance, achieving a MAP@3 of 0.99872**,** accuracy of 0.99744**,** and a low validation loss of 0.019943**.** The increasing MAP@3 and accuracy along with decreasing validation loss indicate effective fine-tuning and strong generalization on the validation set.

6. Performance & Comparative Analysis

6.1. Comparative Results Table

Model Architecture	Type	MAP@3
TF-IDF Neural Network	Choice Model	~0.751
BiLSTM + Attention	From Scratch	~0.74
DeBERTa-v3-small	PreTrained	~0.752

The results indicate a clear hierarchy. The custom TF-IDF NN provides a very strong lexical matching baseline, but the pretrained DeBERTa discriminative transformer significantly outperforms it due to its deep semantic understanding. The scratch model BiLSTM+Attention is computed good increase in hyperparam or tuning it would increase the score at high level.

7. Error Analysis & Failure Mode Diagnostics

Semantically Similar Answer Options: Questions with multiple answer choices containing highly similar wording or closely related concepts often resulted in incorrect predictions, as distinguishing subtle semantic differences remained challenging across all three models

Long and Complex Questions: Questions with lengthy contextual information or requiring multi-step reasoning were more difficult for the models, particularly the TF-IDF baseline, as they demand deeper contextual understanding and semantic reasoning.

8. Conclusion & Future Work

8.1. Key Contributions

This project successfully implemented and evaluated three different approaches for multiple-choice question answering: TF-IDF with Logistic Regression**,** Attention-based BiLSTM, and fine-tuned DeBERTa-v3-small**.** A comprehensive comparison was performed using Accuracy, Macro F1-score, and MAP@3 to analyze the effectiveness of traditional machine learning, recurrent neural networks, and transformer-based architectures. Among the three models, the fine-tuned DeBERTa-v3-small achieved the best overall performance by effectively capturing contextual relationships between the question and answer options. Furthermore, the final model was successfully deployed as an interactive web application using Gradio on Hugging Face Spaces, enabling real-time multiple-choice question prediction.

8.2. Key Learnings

- Traditional Machine Learning**:** TF-IDF combined with Logistic Regression provided a simple yet effective baseline, demonstrating the effectiveness of statistical text representations for text classification tasks.
- Sequential Deep Learning: The Attention-based BiLSTM model effectively captured contextual dependencies within the input sequence, while the attention mechanism improved the model's ability to focus on important words during prediction.
- Transfer Learning: Fine-tuning the pre-trained DeBERTa-v3-small model significantly improved prediction performance by leveraging contextual language representations learned from large-scale text corpora.
- Model Evaluation: The use of multiple evaluation metrics, including Accuracy, Macro F1-score, and MAP@3, provided a comprehensive assessment of model performance and enabled meaningful comparisons between different architectures.

8.3. Challenges Faced

- Label Noise: Empirical analysis revealed systematic label conflicts between the training set and hidden test set for approximately 13 out of 500 test questions, creating an inherent performance ceiling.
- Model Generalization: Selecting appropriate preprocessing techniques, sequence lengths, and hyperparameters was essential to improve the generalization capability of deep learning and transformer-based models.

8.4. Future Work

- Larger Transformer Backbones: Fine-tuning DeBERTa-v3-large or ELECTRA-large with sufficient GPU memory (A100).
- External Knowledge RAG: Building a local Wikipedia FAISS index with larger open-source LLMs (Qwen2.5-14B) for fact verification.
- Hyperparameter Optimization: Perform systematic hyperparameter tuning using automated optimization techniques to identify the most effective training configuration.

9. References

[1] DeBERTaV3: Improving DeBERTa using ELECTRA-style Pre-training with Gradient-Disentangled Embedding Sharing

[2] Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems.

[3] Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.

[4] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.